AristoPay — System Design & Architecture

1. High-Level Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              CLIENT LAYER                                 │
│                                                                           │
│   Browser / Mobile Browser                                                │
│   Next.js 15 (App Router) — Vercel                                        │
│                                                                           │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐                │
│   │ Public   │  │ Admin    │  │ Client   │  │ SP       │                │
│   │ Pages    │  │ Panel    │  │Dashboard │  │Dashboard │                │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘                │
└──────────────────────────────┬────────────────────────────────────────────┘
                               │
                               │
              ┌────────────────┼────────────────┐
              │ REST API       │ WebSocket      │ Firebase Auth
              │ (HTTPS)        │ (Socket.io)    │ (OAuth)
              ▼                ▼                ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                              SERVER LAYER                                 │
│                                                                           │
│   NestJS — AWS (EC2/ECS)                                                  │
│                                                                           │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐                │
│   │  Auth    │  │ Messages │  │ Trustap  │  │ Files    │                │
│   │ Module   │  │ Gateway  │  │ Gateway  │  │ Upload   │                │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘                │
│                                                                           │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐                │
│   │  SP      │  │Subscript │  │ Verif    │  │ Reviews  │                │
│   │ Module   │  │ Module   │  │ Module   │  │ Module   │                │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘                │
└──────────────────────────────┬────────────────────────────────────────────┘
                               │
          ┌────────────────────┼────────────────────┐
          ▼                    ▼                    ▼
┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐
│ PostgreSQL       │  │ Firebase         │  │ Local Disk       │
│ (Prisma ORM)     │  │ (Auth/OAuth)     │  │ uploads/         │
│ AWS RDS          │  │                  │  │ ⚠️ needs S3      │
└──────────────────┘  └──────────────────┘  └──────────────────┘
          │
```

```
        ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                        THIRD-PARTY SERVICES                              │
│                                                                          │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐               │
│   │ Trustap  │  │  Stripe  │  │  Veriff  │  │  SMTP    │               │
│   │ (Escrow) │  │(Payments)│  │ (KYC/ID) │  │  (Mail)  │               │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘               │
└──────────────────────────────────────────────────────────────────────────┘
```

2. Frontend Architecture

Route Structure (Next.js App Router)

```
src/app/
│
├── (main-layout)/              ← Public pages (no auth)
│   ├── (home)/page.tsx         ← Landing page
│   ├── blog/                   ← Blog listing + detail
│   ├── privacy/                ← Privacy policy
│   └── terms/                  ← Terms of service
│
├── (auth-layout)/              ← Auth pages (redirect if logged in)
│   ├── login/                  ← Email login + Google/Apple OAuth
│   ├── register/               ← OTP-based registration
│   ├── sign-up/                ← Role selection
│   └── forgot-password/        ← Password reset
│
├── (admin)/                    ← Admin panel (ADMIN role only)
│   └── admin/
│       ├── dashboard/          ← Stats, revenue chart, recent users
│       ├── user-management/    ← All users, verify SP
│       ├── banner-management/  ← CRUD banners
│       ├── blog-management/    ← CRUD blogs
│       ├── badges-management/  ← CRUD badges
│       ├── subscription-management/ ← CRUD plans
│       └── deal-management/    ← Transactions overview
│
├── (client)/                   ← Client dashboard (CLIENT role)
│   └── client/
│       ├── discover/           ← Browse SP services
│       │   └── [id]/           ← SP profile + Message/Transact
│       ├── messages/           ← Chat with SP (Socket.io)
│       ├── transact/           ← Initiate transaction
│       ├── review-proposals/   ← View/send proposals
│       ├── my-services/        ← Client's active services
│       ├── ratings-rewards/    ← Badges earned
│       └── settings/           ← Profile, payments, history
│
└── (service-provider)/         ← SP dashboard (SERVICE_PROVIDER role)
    └── sp/
        ├── connect/            ← Browse clients
```

```
│   └── [id]/           ← Client profile + Message
├── messages/           ← Chat with client (Socket.io)
├── transact/           ← New transaction + track
├── saved-clients/      ← Favorited clients
├── saved-contracts/    ← Saved DocuSign contracts
├── my-services/        ← SP's service listings
├── ratings-rewards/    ← Badges earned
├── verify-account/     ← Veriff identity verification
└── settings/           ← Profile, payments, history
```

Frontend Layer Architecture

```
┌──────────────────────────────────────────────────────┐
│                    UI Layer                          │
│   Pages (app/) + Components (components/)            │
│   Framer Motion animations                           │
│   Tailwind CSS styling                               │
└──────────────────────┬───────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────┐
│                  Hooks Layer                         │
│   TanStack Query (useQuery / useMutation)            │
│   Custom hooks per domain:                          │
│   use-admin, use-sp, use-auth, use-messages,         │
│   use-subscription, use-verif, use-trustap,          │
│   use-favorites, use-reviews, use-files              │
└──────────────────────┬───────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────┐
│                Services Layer                        │
│   Pure fetch functions per domain:                  │
│   admin-service, sp-service, auth-service,           │
│   messages-service, subscription-service,            │
│   trustap-service, verif-service, files-service      │
│   All unwrap { data } from backend response wrapper │
└──────────────────────┬───────────────────────────────┘
                       │
┌──────────────────────▼───────────────────────────────┐
│                  State Layer                         │
│   Zustand: useAuthStore (user, accessToken,          │
│            hydrated flag)                            │
│   Zustand: useSavedContracts (local contracts)       │
│   TanStack Query cache (server state)                │
└──────────────────────────────────────────────────────┘
```

Auth Guard Flow

```
Route accessed
      │
      ▼
AuthGuard checks hydrated state
(prevents redirect on SSR/refresh)
      │
      ▼
Is user logged in? (Zustand)
      │
   No ──► Redirect to /login
      │
   Yes
      │
      ▼
Has required role?
(ADMIN / CLIENT / SERVICE_PROVIDER)
      │
   No ──► Redirect to appropriate dashboard
      │
   Yes
      │
      ▼
Render page
```

3. Backend Architecture

Module Structure (NestJS)

```
src/
│
├── main/                       ← Core business modules
│   ├── auth/                   ← Registration, Login, OTP, OAuth, JWT
│   ├── users/                  ← User CRUD (admin)
│   ├── service-provider/       ← SP profile CRUD + admin verify
│   ├── subscriptions/          ← Stripe plans + checkout + webhooks
│   ├── badge/                  ← Badge CRUD (admin)
│   ├── banner/                 ← Banner CRUD (admin)
│   ├── blog/                   ← Blog CRUD (admin)
│   └── verif/                  ← Veriff identity verification
│
├── messages/                   ← Real-time messaging
│   ├── messages.controller.ts  ← REST: send, get, mark-read
│   ├── messages.gateway.ts     ← Socket.io: send_message, get_conversation
│   └── messages.service.ts     ← Business logic
│
├── trustap/                    ← Escrow payment
│   ├── trustap.controller.ts   ← REST: create transaction, webhook
│   ├── trustap.gateway.ts      ← Socket.io: transaction events
│   └── trustap.service.ts      ← Trustap API integration
│
├── services/                   ← Service items + proposals
│   └── services.controller.ts  ← CRUD service items, proposals
│
├── reviews/                    ← Ratings & reviews
├── favorites/                  ← Save clients/SPs
│
└── common/                     ← Shared infrastructure
    ├── config/
    │   ├── files/              ← File upload (⚠️ local disk, needs S3)
    │   ├── firebase/           ← Firebase Admin SDK
    │   ├── mail/               ← Nodemailer SMTP
    │   └── prisma/             ← Prisma client
    ├── guards/auth.guard.ts    ← JWT + role-based guard
    ├── response.interceptor.ts ← Wraps all responses: { success, statusCode, message, data }
    └── global-exception.filter.ts ← Global error handler
```

Request/Response Flow

```
HTTP Request
     │
     ▼
AuthGuard (JWT verify + role check)
     │
     ▼
Controller (route handler)
     │
     ▼
Service (business logic + Prisma)
     │
     ▼
ResponseInterceptor
     │
     ▼
{ success, statusCode, message, data }
```

WebSocket Flow (Messages)

```
Client connects to /messages namespace
     │
     ▼
JWT token verified (from auth header / query / handshake.auth)
     │
     ▼
Socket joins room: user:{userId}
     │
     ▼
emit: send_message { recipientId, message }
     │
     ▼
MessagesService.sendMessage()
     │
     ▼
server.to(`user:${recipientId}`).emit('receive_message', msg)
server.to(`user:${recipientId}`).emit('sms_received', msg)
socket.emit('message_sent', msg)
```

4. Database Schema (PostgreSQL + Prisma)

Entity Relationship Overview

```
User (1) ──────────────── (1) ServiceProvider
  │                              │
  │ (1:N)                        │
  ├── UserSubscription ─── SubscriptionPlan
  │
  ├── EscrowTransaction (as buyer)
  ├── EscrowTransaction (as seller)
  │         │
  │         └── ServiceItem (optional)
  │                  │
  │                  └── ServiceReview
  │                  └── ServiceProposal
  │
  ├── SmsMessage (as sender)
  ├── SmsMessage (as recipient)
  │
  ├── UserFavorite (as user)
  ├── UserFavorite (as target)
  │
  ├── ServiceReview (as author)
  └── ServiceReview (as target)
```

Key Models

```
User
├── id (Int, PK)
├── email (unique)
├── name
├── country
├── role (UserRole[]) — ADMIN | CLIENT | SERVICE_PROVIDER
├── passwordHash
├── trustapUserId (unique) — Trustap account link
├── firebaseUid (unique) — OAuth link
├── isEmailVerified
├── isIdentityVerified
├── PROVIDER — PASSWORD | GOOGLE | APPLE
├── verifIdentityVerificationStatus
└── [relations: subscriptions, transactions, messages, favorites, reviews]

ServiceProvider
├── id (Int, PK)
├── userId (unique FK → User)
├── Fullname, occupation, description, location, phoneNumber
├── paymentdetails
└── isVerifiedFromAdmin

EscrowTransaction
├── id (Int, PK)
├── trustapTransactionId (unique)
├── buyerId (FK → User)
├── sellerId (FK → User)
├── serviceItemId (FK → ServiceItem, optional)
├── amount, currency, description
├── status — PENDING | PAYMENT_RECEIVED | IN_ESCROW |
│            COMPLETED | DISPUTED | FUNDS_RELEASED |
│            REFUNDED | CANCELLED
└── paymentUrl

UserSubscription
├── id (Int, PK)
├── userId (FK → User)
├── planId (FK → SubscriptionPlan)
├── stripeSubscriptionId (unique)
├── status — PENDING | ACTIVE | TRIALING | PAST_DUE |
│            INCOMPLETE | CANCELED | UNPAID
```

```
└── currentPeriodStart / currentPeriodEnd

SmsMessage
├── id (String, cuid)
├── senderId (FK → User)
├── recipientId (FK → User)
├── message
├── attachmentType (image | pdf)
├── attachmentUrl
└── read (Boolean)

ServiceItem
├── id (Int, PK)
├── description, industry, location
└── [relations: reviews, escrowTransactions]
⚠️ Missing: userId/serviceProviderId link
```

5. Authentication Architecture

```
┌──────────────────────────────────────────────────────────┐
│              Authentication Methods                      │
│                                                          │
│  Email/Password + OTP          Google / Apple            │
│  ────────────────────          ──────────────────        │
│  POST /auth/register           Firebase SDK              │
│  (sends OTP email)             signInWithPopup()         │
│       │                              │                   │
│  POST /auth/register/verify-otp      │                   │
│       │                              │                   │
│       └──────────────┬───────────────┘                   │
│                      ▼                                   │
│              POST /auth/social-login                     │
│              { firebaseToken, provider }                 │
│                      │                                   │
│                      ▼                                   │
│           Firebase Admin verifyIdToken()                 │
│                      │                                   │
│                      ▼                                   │
│              JWT issued (7 days)                         │
│              Stored in Zustand                           │
└──────────────────────────────────────────────────────────┘
```

6. Payment Architecture (Trustap Escrow)

```
SP initiates transaction
         │
         ▼
POST /trustap/transactions
{ seller_id, buyer_id, price, currency, description }
         │
         ▼
Trustap API creates transaction
Returns: { id, payment_url }
         │
         ▼
EscrowTransaction saved in DB
status: PENDING
         │
         ▼
Client visits payment_url
Pays via Credit Card / ACH
         │
         ▼
Trustap Webhook → POST /trustap/webhook
         │
         ▼
Status updated in DB:
PENDING → PAYMENT_RECEIVED → IN_ESCROW
         │
         ▼
Client confirms delivery
         │
         ▼
Trustap releases funds to SP
Status: FUNDS_RELEASED → COMPLETED
         │
         ▼
SP receives payment
(minus 5% AristoPay fee)
Client paid 3% fee on top
```

7. Subscription Architecture (Stripe)

```
Admin creates plan
POST /subscriptions/admin/plans
         │
         ▼
SubscriptionPlan saved in DB
(name, amount, interval, stripeProductId, stripePriceId)
         │
         ▼
User subscribes
POST /subscriptions/checkout
         │
         ▼
Stripe Checkout Session created
User redirected to Stripe
         │
         ▼
Stripe Webhook → POST /subscriptions/webhook
         │
         ▼
UserSubscription status updated:
PENDING → ACTIVE
         │
         ▼
Premium features unlocked:
- Identity Verification (Veriff)
- Send proposals (Client)
- Receive proposals from verified clients (SP)
- DocuSign contracts
- Transaction Concierge
```

8. Identity Verification Architecture (Veriff)

```
SP/Client wants verification
         │
         ▼
Must have active subscription
         │
         ▼
POST /verif/create-session
         │
         ▼
Veriff API creates session
Returns: { sessionId, verificationUrl }
         │
         ▼
User redirected to Veriff
Completes KYC (ID + selfie)
         │
         ▼
Veriff Webhook → POST /verif/callback
         │
         ▼
Status updated:
CREATED → STARTED → SUBMITTED → APPROVED
         │
         ▼
User.isIdentityVerified = true
User.verifIdentityVerificationStatus = APPROVED
         │
         ▼
Verified badge visible on profile
```

9. File Upload Architecture

```
Current (⚠️ BROKEN in production):
──────────────────────────────────

POST /files/upload-image
         │
         ▼
multer diskStorage
         │
         ▼
Saved to: uploads/images/filename.png
         │
         ▼
URL: {APP_URL}/uploads/images/filename.png
         │
         ▼
⚠️ Lost on every redeploy (no persistence)

Needed Fix:
───────────

POST /files/upload-image
         │
         ▼
multer-s3
         │
         ▼
AWS S3 Bucket
         │
         ▼
URL: https://bucket.s3.region.amazonaws.com/images/filename.png
         │
         ▼
✅ Permanent, CDN-ready
```

10. API Endpoints Summary

Auth

Method	Endpoint	Auth	Description
POST	/auth/register	❌	Register + send OTP
POST	/auth/register/verify-otp	❌	Verify OTP
POST	/auth/login	❌	Email login
POST	/auth/social-login	❌	Google/Apple OAuth
POST	/auth/forgot-password	❌	Send reset email
POST	/auth/reset-password	❌	Reset password
GET	/auth/me	✅	Get current user

Messages

Method	Endpoint	Auth	Description
POST	/messages/send	✅	Send message
GET	/messages/conversation	✅	All conversations
GET	/messages/conversation/:id	✅	Single conversation
PATCH	/messages/:id/read	✅	Mark as read

Trustap

Method	Endpoint	Auth	Description
POST	/trustap/transactions	✅	Create transaction
GET	/trustap/transactions	✅	Get transactions
POST	/trustap/webhook	❌	Trustap webhook

Subscriptions

Method	Endpoint	Auth	Description
GET	/subscriptions/plans	❌	All plans
POST	/subscriptions/checkout	✅	Create checkout
GET	/subscriptions/my	✅	My subscriptions
POST	/subscriptions/admin/plans	✅ ADMIN	Create plan
PATCH	/subscriptions/admin/plans/:id	✅ ADMIN	Update plan
GET	/subscriptions/admin/finance-summary	✅ ADMIN	Finance stats

Service Provider

Method	Endpoint	Auth	Description
POST	/service-provider/create-service-provider	✅	Create SP profile
GET	/service-provider/all-service-providers	✅	All SPs
GET	/service-provider/:id	✅	Single SP
PATCH	/service-provider/service-providers/:id	✅ ADMIN	Verify SP

11. Environment Variables

Frontend (.env.local)

```
NEXT_PUBLIC_API_URL=https://your-backend.com
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
```

Backend (.env)

```
PORT=4000
JWT_SECRET=
JWT_EXPIRES_IN=7d
APP_URL=https://your-backend.com        ← ⚠️ Must be set correctly

DATABASE_URL=postgresql://...

SMTP_HOST=
SMTP_PORT=
SMTP_SECURE=

TRUSTAP_API_KEY=
TRUSTAP_BASE_URL=
TRUSTAP_CLIENT_ID=
TRUSTAP_CLIENT_SECRET=
TRUSTAP_REDIRECT_URI=

STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=

VERIFF_API_KEY=
VERIFF_SHARED_SECRET=
VERIFF_BASE_URL=

FIREBASE_PROJECT_ID=

# Needed (not yet added):
AWS_ACCESS_KEY_ID=          ← For S3
AWS_SECRET_ACCESS_KEY=      ← For S3
AWS_S3_BUCKET_NAME=         ← For S3
AWS_REGION=                 ← For S3
```

12. Known Issues & Technical Debt

Issue	Severity	Status
File upload uses local disk — lost on redeploy	🔴 Critical	Needs S3 migration

Issue	Severity	Status
ServiceItem has no userId/SP link	🔴 Critical	Backend fix needed
Client proposal-sending not enforced at backend	🔴 Critical	Security gap
/sp/connect page uses mock data	🟡 Medium	Needs real API
DocuSign not implemented	🟡 Medium	Future feature
APP_URL env var not set in production	🔴 Critical	Causes broken image URLs
SP PROVIDER field always saves as PASSWORD	🟡 Medium	Backend bug